

Research Note — LLD Practice Platform

Research Note — LLD Practice Platform

1. Learner problem

Low-level design practice is easy to **start** and hard to **evaluate**. A learner can invent classes for a Parking Lot or Elevator in an evening, then still not know whether responsibilities are cohesive, whether coupling is accidental, or whether the design would survive a realistic requirement change.

Today the common paths are:

1. Study a reference solution (GitHub repos, blog posts, “Grokking the LLD Interview” style courses) and compare class names.
2. Self-review against SOLID slogans without concrete evidence from the attempt.
3. Ask a peer or interviewer — high signal, not available on demand.

The failure mode of (1) is especially harmful: two valid designs can look very different, so matching a canonical class list teaches imitation rather than judgement. The failure mode of (2) is vagueness — “I should use Strategy” without pointing at a concrete responsibility smell in the learner’s own write-up.

What learners need for a meaningful attempt is not a blank essay. They need enough **evidence of design thinking**: assumptions, named responsibilities, collaborations, trade-offs, and a short plan for an extension. What they need from feedback is not a single 0–100 score. They need criterion-level comments that cite their own text and suggest the next improvement.

2. Existing approaches (brief survey)

Source / approach	Workflow	Submission	Feedback	Gap

Educative / Grokking-style LLD courses	Read patterns !’ see a reference design	Mostly reading + quizzes		
Reference solution, not personal rubric	Little retry history on <i>*your*</i> design			
InterviewBit / LeetCode Discuss LLD threads	Post a design; hope for comments	Text / UML images	Ad-hoc peer comments	Inconsistent, slow, not structured
Public GitHub “parking-lot” / “elevator” repos	Clone & study	Code	Implicit “this is the answer”	Rewards matching one structure
Diagram-only practice (Excalidraw / Mermaid)	Draw boxes	Diagram	Usually none	Structure without evaluation
Generic ChatGPT “rate my design”	Paste text once	Free-form	Unstable score, weak evidence	No persist/ history/state machine

Patterns across these tools: practice is plentiful; personal, evidence-linked, repeatable evaluation is rare.

3. Product direction

Build a narrow practice loop, not an LMS:

Choose problem !’ Structured text design !’ Submit !’ Rubric evaluation !’ Review !’ Try again (history preserved).

MVP choices:

- Four seeded problems (Parking Lot, Elevator, Vending Machine, Expense Splitter) with constraints and an extension prompt.
- Structured text as the submission format — smallest format that still evidences responsibilities and trade-offs; diagrams/code can plug in later behind the same Submission contract.
- Fixed rubric (requirement coverage, responsibilities, coupling/cohesion, extensibility, explanation) with score, evidence, concern, suggestion, confidence.
- Deterministic validation first; LLM (or heuristic fallback) only for judgment-heavy criteria.
- Persist submission before evaluation; async states draft !’ submitted !’ evaluating !’ completed failed.

This targets the learner problem directly: repeated attempts with comparable, explainable feedback, without pretending there is only one correct LLD.

4. Out of scope (intentionally)

Auth/SSO, multi-tenant LMS features, UML canvas, compiling user code, microservices, and large-scale HLD.

Those do not improve the core practice loop for a focused prototype.